PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

DELIVERABLE 1 – SYNOPSIS / PROJECT PROPOSAL

TaskFlow

A Collaborative Task Manager Application for Small Teams

Item	Student Entry
Project ID	TM-2026-02
Group Number	Team 2
Assigned Project	Task Manager Application (Complexity: Medium)
Team Members	Ankita Pramod – PES1202400201 / Ashritha Kamalakar Hegde – PES1202403359 / B Nuthan – PES1202402203 / Chaithanya B – PES1202403503
Course	UE24CS341A – Software Engineering
Semester	V Semester, B.Tech CSE
Technology	JavaScript – React.js (frontend) + Node.js/Express (backend), PostgreSQL
Development Methodology	Agile (Scrum, 6 sprints)
Version / Date	1.0 / 02 September 2026
Submission Format	PDF unless otherwise instructed

DELIVERABLE 1 – SYNOPSIS / PROJECT PROPOSAL

This proposal covers the project title and group, team profile, proposed project description, functional features, plan of work and clear product ownership, as required by the official synopsis guidelines.

1.1 Project Title and Group

Field	Entry
Project Title	TaskFlow – Collaborative Task Manager Application for Small Teams
Group Number	Team 2
Project ID	TM-2026-02
Customer / Stakeholder	Student project teams and club coordinators at PES University (the team itself is the first customer)

1.2 Team Profile and Product Ownership

Sl.	Name	USN	SRN / PRN	Owned functionality (primary)	Secondary responsibilities
1	Ankita Pramod	PES1UG24CS065	PES1202400201	F1 User accounts, authentication, roles (RBAC) and F8 administration	API contract review, CI/CD (Jenkins, Docker)
2	Ashritha Kamalakar Hegde	PES1UG24CS089	PES1202403359	F2 Task lifecycle (CRUD, priority, status workflow, dependencies) and F3 tags, search and filtering	UI/UX design
3	B Nuthan	PES1UG24CS106	PES1202402203	F4 Projects and team membership and F5 task assignment, comments and activity log	Database design
4	Chaithanya B	PES1UG24CS123	PES1202403503	F6 Reminders and notifications and F7 dashboard and progress reports	Testing/CI, SonarQube

Important: UI design, database interaction and unit testing are supporting responsibilities, not substitutes for owning a functional feature. Every member owns a complete feature end-to-end (API, business logic, UI and unit tests). Requirements, architecture, code reviews and documentation are shared by all four members, and every pull request is reviewed by another member before merge.

1.3 Problem Statement

Small teams (student project groups, club committees, hackathon teams) coordinate work through a mix of WhatsApp messages, shared spreadsheets and memory. This creates recurring problems: nobody has a single view of who is doing what, deadlines are missed because reminders depend on someone remembering to send them, task status is only known by asking, and there is no record of what changed or who did it.

Commercial tools such as Jira or Asana solve this for enterprises but are heavyweight and over-configured for a team of four to ten people.

TaskFlow is a lightweight web application that gives a small team one place to create tasks, assign them, track their status through a defined workflow, get reminded before deadlines and see progress at a glance. It is intentionally scoped to the features a small team actually uses, so it can be learned in minutes rather than days.

Scenario: A project owner creates a project and invites members. Members are assigned tasks with a priority and due date, move them through To Do → In Progress → In Review → Done, and comment on them. The system reminds members before a task falls due, and the owner watches overall progress, overdue work and per-member workload on a dashboard. An administrator manages user accounts and views the audit log.

1.4 Customer, Stakeholders and Intended Users

Actor	Goal / Need	Major actions
Team Member (registered user)	Know what I have to do and by when; update my progress	Register/login, view own tasks, change status, comment, receive reminders
Project Owner (team lead)	Plan and distribute work; see whether the project is on track	Create project, invite members, create/assign tasks, set priority and due date, view dashboard and reports
Administrator	Keep the system healthy and safe	Manage user accounts, deactivate users, view audit log
E-mail Service (external / mock)	Deliver reminders reliably	Send due-date reminder e-mails; in-app notifications are delivered by the system itself

Team Members and Project Owners are non-technical, frequent users accessing the system from laptops and mobile browsers, so the interface must be simple and responsive. The Administrator is a trained, low-frequency user with elevated privileges.

1.5 Objectives

- Provide a single, accurate record of every task: owner, priority, due date, status and history.
- Reduce missed deadlines by sending reminders automatically before a task falls due.
- Give project owners a real-time view of progress, overdue work and member workload without asking around.
- Enforce a defined task workflow (To Do → In Progress → In Review → Done) with role-based rules on who can move a task.
- Make the system secure by design: hashed passwords, token-based authentication, server-side role checks and input validation.
- Develop the product incrementally using Agile practices with full traceability from requirement to design, code, test and validation.

1.6 Scope

In Scope	Out of Scope
User registration, login, profile, roles (Member / Owner / Admin)	Single sign-on with Google/Microsoft accounts
Projects with team membership and invitations	Billing, subscriptions or multi-tenant SaaS hosting
Task CRUD with priority, due date, tags and status workflow	Gantt charts and critical-path scheduling
Task assignment, comments and activity log	Real-time chat / video calls
Task dependencies (blocked-by) and Kanban + list views	Native mobile apps (responsive web only)
Due-date reminders (in-app + e-mail via sandbox SMTP)	Push notifications to mobile devices
Dashboard, progress and workload reports, CSV export	AI-based effort estimation or auto-scheduling
Admin user management and audit log	Integration with external tools (Slack, GitHub issues)

1.7 Functional Features

Feature IDs (F1–F8) are used consistently in the SRS, backlog, traceability matrix and ownership matrix.

ID	Feature	What the user can do	Owner
F1	User accounts and authentication	Register with a unique e-mail, log in and log out, update profile and password; system issues JWT tokens and enforces Member / Owner / Admin roles on every API call	Ankita
F2	Task lifecycle management	Create, view, edit and delete tasks with title, description, priority (Low/Medium/High/Critical), due date and estimate; move tasks through To Do → In Progress → In Review → Done with validated transitions; declare blocked-by dependencies	Ashritha
F3	Tagging, search and filtering	Attach tags to tasks; search by keyword; filter and sort by status, priority, assignee, tag and due date; switch between Kanban board and list view	Ashritha
F4	Projects and team membership	Create a project, invite members by e-mail, accept/decline invitations, assign project roles, archive a project	Nuthan
F5	Assignment, comments and activity log	Assign or re-assign a task to a member, comment on a task with @mentions, view a chronological activity log of every change to the task	Nuthan
F6	Reminders and notifications	Receive in-app notifications for assignment, mentions and status changes; receive due-date reminders (1 day / 3 days before) by e-mail; mark notifications as read	Chaithanya
F7	Dashboard and progress reports	Personal “Today” view (due today, overdue, in review); project dashboard with completion %, burndown by sprint, overdue count and per-member workload; export a project report to CSV	Chaithanya
F8	Administration	Admin lists, deactivates and reactivates users, resets a user’s role, and views a system-wide audit log of security-relevant events (failed logins, role changes)	Ankita

Differentiation from a basic to-do list: an enforced workflow with role rules (a task cannot be marked Done while it has an open blocking dependency), a workload view so the owner can spot overload before a deadline slips, a full activity log per task, and a personal "Today" view that aggregates what is due across all of a member's projects.

1.8 Short-Term Plan

The project runs as six two-week Scrum sprints. Each sprint ends with a working increment, a demo, a retrospective and updated documentation. Sprints 1–3 are the immediate plan; later sprints are indicative and will be refined in sprint planning.

Iteration	Planned output	Acceptance evidence
Sprint 1	Project setup (repo, branch strategy, Jenkins skeleton, Docker, DB schema v1), SRS baseline, F1 registration/login	User stories + Git repo + working login
Sprint 2	F2 task CRUD + status workflow; F4 projects and membership; architecture + UML	Demo + unit tests + design docs
Sprint 3	F3 tags/search/filter with Kanban view; F5 assignment, comments, activity log; RBAC on all endpoints	Integration tests + PRs + Postman collection
Sprint 4	F6 notifications + e-mail reminders; F7 Today view and project dashboard	System demo + test evidence + coverage ≥ 80 % lines
Sprint 5	F8 administration + audit log; security hardening; SonarQube clean-up; performance test	Validation + coverage + security report
Sprint 6	System validation with pilot users, defect fixing, maintenance plan, final report and presentation	Final report + deployment/demo

1.9 Proposed Technology / Tool Stack

Activity	Tool / technology	Evidence to submit
Frontend	React.js 18, React Router, Axios, Tailwind CSS	Screens for every use case with captions
Backend / API	Node.js 20, Express, REST + JSON, JWT, bcrypt	API documentation (OpenAPI) with inputs, outputs, errors and auth
Database	PostgreSQL 16 with Prisma ORM; migrations in version control	Schema / field layouts, ER diagram
Scheduler / e-mail	node-cron; Nodemailer with Mailtrap sandbox	Reminder test evidence

Activity	Tool / technology	Evidence to submit
Version control	Git + GitHub (feature branches, protected main)	Repository URL, commit history per member, PRs with reviews
Agile planning	Jira (Scrum board) linked to GitHub commits	Backlog, user stories, sprint history
CI/CD	Jenkins: lint → unit tests → coverage → SonarQube → Docker build → deploy to test	Pipeline logs / screenshots per sprint
Containerisation	Docker + docker-compose (api, web, db)	Dockerfile, reproducible run evidence
Testing	Jest + Supertest (API), React Testing Library (UI)	Test reports, line and branch coverage
Static analysis / security	SonarQube, ESLint, npm audit	Analysis and security validation reports
API testing	Postman collection	Collection + run results
Design	draw.io / PlantUML	Use case, class, sequence, activity, component, deployment diagrams

1.10 Agile Justification

Scrum is chosen because the features are largely independent and can be delivered as vertical slices, one per member per sprint, each producing a testable increment. Requirements for a task manager are well understood at the start, but the details of workflow rules, notification timing and dashboard content are best refined from feedback on a working increment rather than fixed up front. Two-week sprints with a demo and retrospective give the course's regular evaluations something concrete to assess, and the Jira board, sprint history and PR reviews provide the continuous evidence of Agile practice that the guidelines require. Definition of Done for every story: code merged via a reviewed PR, unit tests passing in Jenkins, coverage not reduced, requirement ID referenced in the PR, and the relevant document updated.

1.11 Qualitative Properties Each Member Will Deliver

Member	Qualitative property (measurable)
Ankita Pramod	Security: no plaintext passwords; every non-public endpoint rejects missing/expired/invalid tokens and wrong roles (verified by automated authorization tests).
Ashritha Kamalakar Hegde	Usability: a new user can create a project and its first task in no more than 4 screens; search/filter results return within 1 s for 1,000 tasks.
B Nuthan	Data integrity: no orphan tasks, duplicate memberships or invalid state transitions can be created (enforced by DB constraints and negative integration tests).
Chaithanya B	Reliability and performance: reminders are sent exactly once per task per configured interval, and the dashboard renders within 2 s for a project of 500 tasks under the defined test load.